Rappels sur la complexité & analyse asymptotique

Amina AMARA

Maitre assistante, FSS, Université de Sfax

amina.amara@fss.usf.tn

Introduction et définitions

❑L'algorithmique?

Définition (informelle)

Un algorithme est la composition d'un ensemble fini d'étapes, chaque étape étant formée d'un nombre fini d'opérations dont chacune est :

- définie de façon rigoureuse et non ambigue ;
- effective (i.e. pouvant être réalisée en un temps fini).

La notion d'algorithme est plus générale que celle de programme (indépendant du langage de programmation utilisé).

Introduction et définitions

❑L'algorithmique?

Un peu d'histoire

Le mot algorithme vient du nom du mathématicien Al Khwarizmi (820) : introduction de la numérotation décimale et des calculs s'y rapportant.

Introduction et définitions

❑Des besoins contradictoires!

Un algorithme doit :

- être simple à comprendre, à mettre en oeuvre et à mettre au point ;
- mettre intelligement à contribution les ressources de l'ordinateur, et plus précisément, il doit s'exécuter rapidement.

Introduction et définitions

❑Des besoins contradictoires!

Pourquoi ces besoins sont contradictoires ?

•Un algorithme **très simple** est souvent **moins performant** (plus lent).

•Un algorithme **très rapide** est souvent **plus complexe**, donc :

- plus difficile à comprendre,
- plus difficile à programmer et à déboguer.

Introduction et définitions

❑C’est quoi un **ALGORITHME**?

Le crumble aux pommes du chat qui tousse · Dessert · Végétarien

Très facile €

Ingrédients (pour 6 personnes) :

- 6 belles pommes (des Canada par exemple)
- 250 g de cassonnade
- 150 g de farine
- 125 g de beurre (le sortir 1/2 heure avant de commencer la recette)
- le jus d'un citron
- une petite cuillère de cannelle en poudre
- 1 sachet de sucre vanillé

Préparation :

Préchauffer le four à thermostat 7 (210°C).

Peler, évider et découper les pommes en cubes grossiers, les répartir dans un plat allant au four, verser dessus le jus du citron, la canelle et le sucre vanillé.

Dans un saladier, mélanger la farine et la cassonade.
Puis ajouter le beurre en petits cubes et mélanger à la main de façon à former une pâte grumeleuse.

Emietter cette pâte au dessus des pommes de façon à les recouvrir.
Mettre au four une bonne 1/2 heure.

Servir tiède avec de la crème fouettée ou de la glace à la vanille.

Un problème

Des données

Des instructions

Un résultat

Classiquement assimilable a une recette de cuisine

Introduction et définitions

❑Algorithme : définitions

Un algorithme =

- Description précise des opérations à faire pour résoudre un problème (suite d'instructions).
- Procédure de calcul bien définie qui prend en entrée une valeur, ou un ensemble de valeurs et qui donne en sortie une valeur, ou un ensemble de valeurs.

Introduction et définitions

❑Algorithme : définitions

Un *bon* algorithme =

- Un algorithme correct : i.e. pour chaque instance en entrée, l'algorithme se termine en produisant la bonne sortie
  $\Rightarrow$ Savoir prouver un algorithme
- Un algorithme efficace : mesure de la durée que met un algorithme pour produire un résultat
  $\Rightarrow$ Savoir analyser la complexité d'un algorithme : i.e. détermination de l'espace mémoire et du temps d'exécution nécéssaire à la résolution du problème.

Pour un problème donne, plusieurs algorithmes ou aucun sont possibles.
Un algorithme se termine en un temps fini.

Introduction et définitions

- Un exemple simple
  - Problème:
    - **Enoncé**: Ecrire une fonction qui renvoie **1** si un nombre **n** appartient à une liste **L** ou 0 si non.

Introduction et définitions

❑Un exemple simple

❑Solutions proposées (Si on considère le langage de programmation Python):

- Plusieurs solutions sont possibles pour répondre à cet exercice.
  - ➢ L'opérateur **in**
  - ➢ La boucle **for**
  - ➢ La boucle **while**
  - ➢ Fonction prédéfinie **count**
  - ➢ Fonction recursive
  - ➢ Ect

Introduction et définitions

❑Un exemple simple

❑Solutions proposées (Si on considère le langage de programmation Python):

(1)

```
def chercher(n, L):
        if n in L :
                return 1
        return 0
```

(2)

```
def chercher(n, L):
        for i in range(0, len(L)):
                if L[i]==n :
                        return 1
        return 0
```

(3)

```
def chercher(n, L):
        i = 0
        while i<len(L) :
                if L[i]==n :
                        return 1
                i=i+1
        return 0
```

(4

```
def chercher(n, L):
        if L= =[] :
                return 0
        if L[0]==n:
                return 1
        return chercher(n, L[1:])
```

(5)

```
def chercher(n, L):
        if L.count(n)>=1 :
                return 1
        return 0
```

(6)

```
def chercher(n, L):
        if L==[] :
                return 0
        if L[0]==n:
                return 1
        return chercher(n, L[1:])
```

Introduction et définitions

❑Quelques Questions

- ➢Le code le plus courts: est-il meilleur ?
- ➢Comment peut on désigner le meilleur code parmi les solutions proposées ?

Complexité Algorithmique

❑Théorie de la complexité algorithmique

- La **théorie de la complexité** est un domaine des mathématiques, et plus précisément de l'informatique théorique, qui étudie formellement la *quantité de ressources* (temps et/ou espace mémoire) nécessaire pour résoudre un *problème algorithmique* au moyen de l'exécution d'un algorithme.

Complexité Algorithmique

❑Objectifs des calculs de complexité

- ➢pouvoir prévoir le temps d'exécution d'un algorithme
- ➢pouvoir comparer deux algorithmes réalisant le même traitement

- Exemples :
  - ➢si on lance le calcul de la factorielle de 100, combien de temps faudra t-il attendre le résultat?
  - ➢quel algorithme de tri vaut-il mieux utiliser pour retrier un tableau où on vient de changer un élément?

Complexité Algorithmique

❑Types de complexité

Deux types de compléxité :

- **En espace** : Quelle quantité de place en mémoire va t-on utiliser ?
- **En temps** : Combien d'opérations va t-on effectuer ?

Complexité Temporelle VS Complexité Spatiale

❑Types de complexité

- L'efficacité d'un algorithme peut être évalué en temps et en espace :
  - **Complexité en temps** : évaluation du temps d'exécution de l'algorithme
  - **Complexité en espace** : évaluation de l'espace mémoire occupé par l'exécution de l'algorithme

  - Règle (non officielle) de l'espace-temps informatique : pour gagner du temps de calcul, on doit utiliser davantage d'espace mémoire.
  - On s'intéresse essentiellement à la complexité en temps (ce qui n'était pas forcément le cas quand les mémoires coutaient cher)

Complexité Temporelle VS Complexité Spatiale

❑Types de complexité

- **Exemple:** échange de deux valeurs entières

```
// échange des valeurs de deux variables
entier x, y, z;
... // initialisation de x et y
z ← x;
x ← y;
y ← z;
```

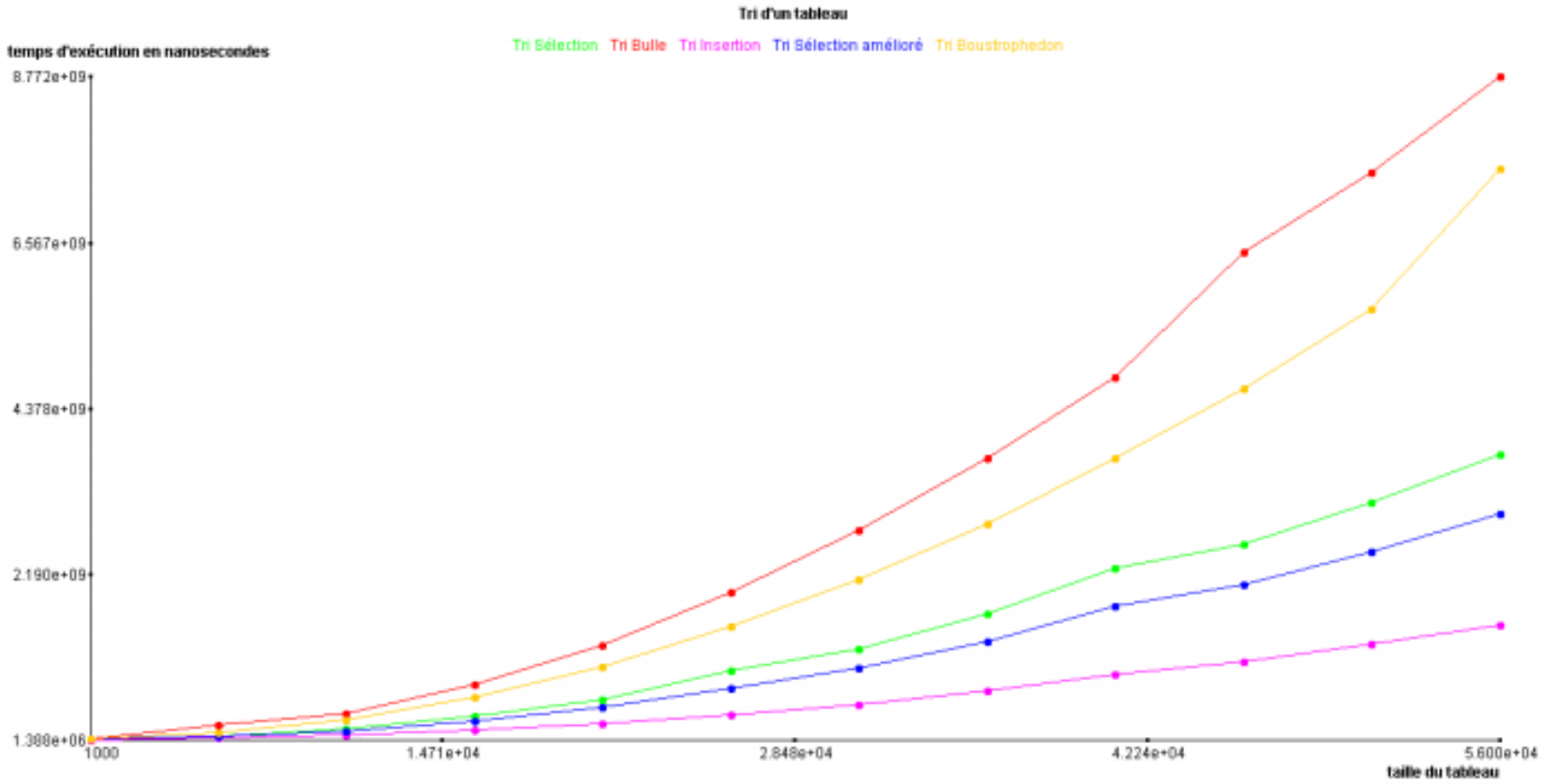
```
// échange des valeurs de deux variables
entier x, y;
... // initialisation de x et y
x ← y-x;
y ← y-x;
x ← y+x;
```

- La première méthode utilise une variable supplémentaire et réalise 3 affectations
- La deuxième méthode n'utilise que les deux variables dont on veut échanger les valeurs, mais réalise 3 affectations et 3 opérations

Complexité Temporelle VS Complexité Spatiale

❑Types de complexité

➢Il est possible d'évaluer de façon expérimentale le **temps d'exécution** des programmes.



Complexité Temporelle VS Complexité Spatiale

❑Types de complexité

➢Attention!!!!

Le temps d'exécution

Le temps d'exécution est la durée (en secondes, minutes, etc.) nécessaire au programme pour s'éxecuter.

Mais le temps d'éxecution dépend :

- de la machine ;
- du système d'exploitation ;
- du langage ;
- de la donnée ;

On veut une méthode **indépendante de l'environnement**.

Complexité Temporelle VS Complexité Spatiale

- Types de complexité

Comparer les algorithmes théoriquement

&

Pas besoin d'implémentation

Calcul de la complexité

❑Etapes de calcul de la complexité

1. Identifier le **paramètre** de complexité de l'algorithme
2. Calculer la **complexité** en fonction du **paramètre** de la complexité
3. Calculer la **complexité asymptotique**

Calcul de la complexité

❑Etapes de calcul de la complexité

1. Identifier le paramètre de complexité de l'algorithme

Le **paramètre de complexité** est la **quantité qui mesure la "taille" des données du problème** et sur laquelle dépend le temps ou la mémoire nécessaires pour résoudre le problème.

❖ **Exemples:**

- ✓ Somme d'une liste : **n** = nombre d'éléments.
- ✓ Recherche dans un tableau : **n** = taille du tableau.
- ✓ Tri d'un tableau : **n** = nombre d'éléments à trier.
- ✓ Multiplication de matrices : **n** = dimension des matrices (n×n).

Calcul de la complexité

❑Etapes de calcul de la complexité

2. Calculer la **complexité** en fonction du **paramètre** de la complexité

Pour calculer la complexité, on compte le **nombre d'opérations élémentaires** (affectations, comparaisons, additions…) effectuées par l'algorithme en fonction du paramètre de complexité **n**.

Cela permet d'exprimer le coût total en fonction de la taille du problème.

Calcul de la complexité

❑Etapes de calcul de la complexité

2. Calculer la complexité en fonction du paramètre de la complexité

Opération élémentaire

Une opération élementaire est une opération qui prend un temps constant

- Même temps d'exécution quelque soit la donnée

Exemples d'opérations en temps constant

- Instructions assembleur
- Opérations arithmétiques ($+, \times, -$), affectation, comparaisons sur les **types primitifs** (entiers, flottants, etc.)

Calcul de la complexité

❑Etapes de calcul de la complexité

2. Calculer la complexité en fonction du paramètre de la complexité

❖Exemple 1

L'algorithme suivant calcule $n! = n \times (n-1) \times (n-2) \times \cdots \times 2 \times 1$ (avec $0! = 1$).

Algorithme : `Factorielle`

Données : un entier n

Résultat : un entier valant $n!$

		nombre	coût
1	$fact \leftarrow 1$;		
2	**pour** i *allant de* 2 *à* n **faire**		
3	$\quad fact \leftarrow fact * i$;		
4	**retourner** $fact$;		

Nombre total d'opérations :

Calcul de la complexité

❑Etapes de calcul de la complexité

2. Calculer la **complexité** en fonction du **paramètre** de la complexité

❖Exemple 1

L'algorithme suivant calcule $n! = n \times (n-1) \times (n-2) \times \cdots \times 2 \times 1$ (avec $0! = 1$).

Algorithme : `Factorielle`

Données : un entier n

Résultat : un entier valant $n!$

```
1 fact ← 1;
2 pour i allant de 2 à n faire
3     fact ← fact * i;
4 retourner fact;
```

	nombre	coût
initialisation :	$1\times$	1 op.
itérations :	$n\times$	1 op.
mult. + affect. :	$(n-1)\times$	2 op.
retour fonction :	$1\times$	1 op.

Nombre total d'opérations :

$$1 + n + (n-1) * 2 + 1 = 3n$$

Calcul de la complexité

❑Etapes de calcul de la complexité

Règles générales utilisées pour le décompte

❖Séquence d'instructions

$$T_{séquence}(n) = \sum T_{éléments\ de\ la\ séquence}(n)$$

❖Branchements conditionnels

$$T(n) = T_C(n) + \max\{T_J(n), T_K(n)\}$$

Calcul de la complexité

❑Etapes de calcul de la complexité

Règles générales utilisées pour le décompte

- ❖ Boucles
  - ❖ Itération bornée : pour i de j à k faire B.

$$T(n) = (k - j + 1).\left(T_{entête}(n) + T_B(n)\right) + T_{entête}(n)$$

Dans l'en-tête est mis l'affectation de l'indice et le test de continuation.

  - ❖ Itération non bornée :
    - ❖ tant que C faire B

$$T(n) = Nb_{boucles}\,.\left(T_B(n) + T_C(n)\right) + T_C(n)$$

      - ❖ avec $\boldsymbol{Nb_{boucles}}$ le nombre de boucles qui s'évalue
    - ❖ Répéter B jusqu'à C

$$T(n) = Nb_{boucles}.\left(T_B(n) + T_C(n)\right)$$

Calcul de la complexité

❑Etapes de calcul de la complexité

❑Exemple

Blocs d'instructions	Coût
X = a + b	
A = 0 B = 1 X = A + B	
A = X + Y + Z	
if A == B : pirnt("Egalité") else : c = A – B print(c)	
S = 0 for i in range(1, 11) : print("Donner A : ") A = input() S = S + A	

Blocs d'nstructions	Coût
for i in range(1, 11) : print("table de : " , i) for j in range(1, 11) : print(i, " x ", j , " = " , i*j)	
for i in range(1, 5) : for j in range(1, i + 1) : print("X", end= '') print("")	

Nuances de la complexité temporelle

- Lorsque, pour une valeur donnée du paramètre de complexité, le temps d'exécution varie selon les données d'entrée, on peut distinguer:
  - **La complexité au pire**: temps d'exécution maximum, dans le cas le plus défavorable
  - **La complexité au mieux**: temps d'exécution minimum, dans le cas le plus favorable (en pratique cette complexité n'est pas très utile)
  - **La complexité moyenne**: temps d'exécution dans un cas médian, ou moyenne des temps d'exécution

Le plus souvent, on utilise la complexité au pire, car on veut borner le temps d'exécution

Nuances de la complexité temporelle

- Exemple: calculer la complexité aux mieux, moyenne et pire de cas

```
def rechercherElement(L , x) :
        i=0
        taille =  len(L)
        while i < taille :
                if L[i]==x :
                    return True
                i = i + 1
        return False
```

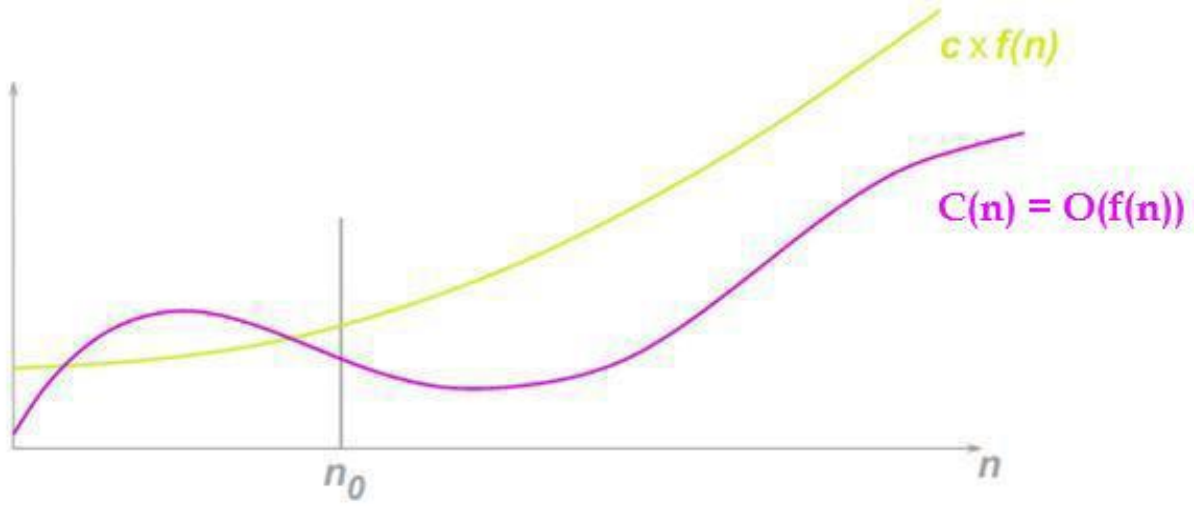
Complexité Asymptotique: Notation Big-O (de Landau)

- Ce qui nous intéresse n'est pas un *temps précis d'exécution* mais un ordre de grandeur de ce temps d'exécution en fonction de la taille des données.
  - **Première approximation** : Considérer la complexité au pire des cas
  - **Deuxième approximation** : Considérer le calcul asymptotique de la complexité, s'intéresser juste au grandes quantités des données.

Complexité Asymptotique: Notation Big-O (de Landau)

- Soit C(n) une fonction qui désigne le temps de calcul d'un algorithme A.
- On dit que $C(n)$ est en grand O de $f(n)$: $\mathrm{C(n)} = \mathrm{O}(f(n))$

  si et seulement si : $\exists(n_0, \mathrm{c})$ telle que $C(n) \leq c \times f(n)$pour tous $n \geq n_0$



La notation O(f (n)) est aussi appelée Notation de Landau : (Complexité asymptotique) (i.e.quand n → ∞)

On dit que *f* est une borne supérieure asymptotique pour *C*.
Vite dit : *C est dépassée par f à partir d'une certaine taille de données*

Complexité Asymptotique: Notation Big-O (de Landau)

- Exemple 1:
  - Soit $C(n) = 3n + 6$.
  - Montrer que $f(n) = n$ est une borne supérieure pour $C(n)$

Complexité Asymptotique: Notation Big-O (de Landau)

- **Exemple 1**: Si $C(n) = 3n + 6$ alors $C(n) = O(n)$
  - Démonstration :
    - On cherche c tel que : $3n + 6 \leq c \cdot n$ pour $n \geq n_0$
    - On résout l'inégalité : $3n + 6 \leq c \cdot n$

      $6 \leq (c - 3)n$

      $c - 3 \geq 6 / n$
    - Pour $n_0 = 2$: $c - 3 \geq 6 / 2 = 3$

      $c \geq 6$
    - On peut choisir n'importe quelle constante ≥ 6, par exemple $c = 9$.

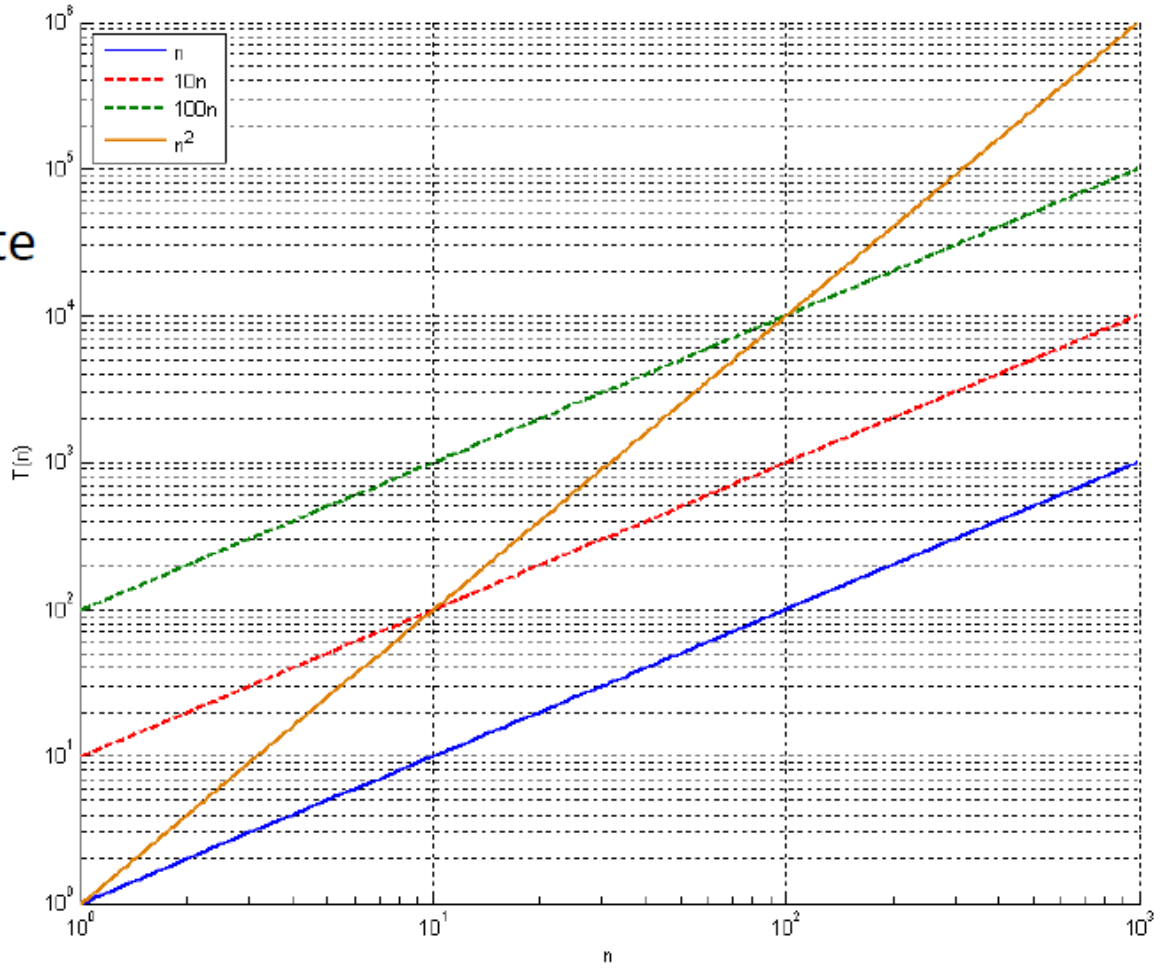
Complexité Asymptotique: Notation Big-O (de Landau)

- Exemple 2: La fonction $\boldsymbol{n^2}$ est-ce qu'elle est O(n)?

Complexité Asymptotique: Notation Big-O (de Landau)

❑Exemple 2: la fonction n^2 n'est pas $O(n)$

- $n^2 \leq c \times n$
- $n \leq c$
- L'inéquation précédente ne peut pas être satisfaite car c doit être une constante



Complexité Asymptotique: Notation Big-O (de Landau)

- Notation O : Simplification
  - On calcule le temps d'exécution comme avant, mais on effectue les simplifications suivantes :
    - On oublie les constantes multiplicatives (elles valent 1) ;
    - On annule les constantes additives ;
    - On ne retient que les termes dominants ;

- Soit un algorithme effectuant $C(n) = 4n^3 - 5n^2 + 2n + 3$ opérations ;

  ⇒On remplace les constantes multiplicatives par 1 : $1n^3 - 1n^2 + 1n + 3$

  ⇒On annule les constantes additives : $n^3 - n^2 + n + 0$

  ⇒On garde le terme de plus haut degré : n^3

  Donc : $C(n) = O(n^3)$

Complexité Asymptotique: Notation Big-O (de Landau)

- Notation O :Propriétés

$$c * O(f(n)) = O(f(n))$$

$$O(f(n)) + O(g(n)) = O(f(n) + g(n))$$

$$O(f(n)) * O(g(n)) = O(f(n) * g(n))$$

Complexité Asymptotique: Notation Big-O (de Landau)

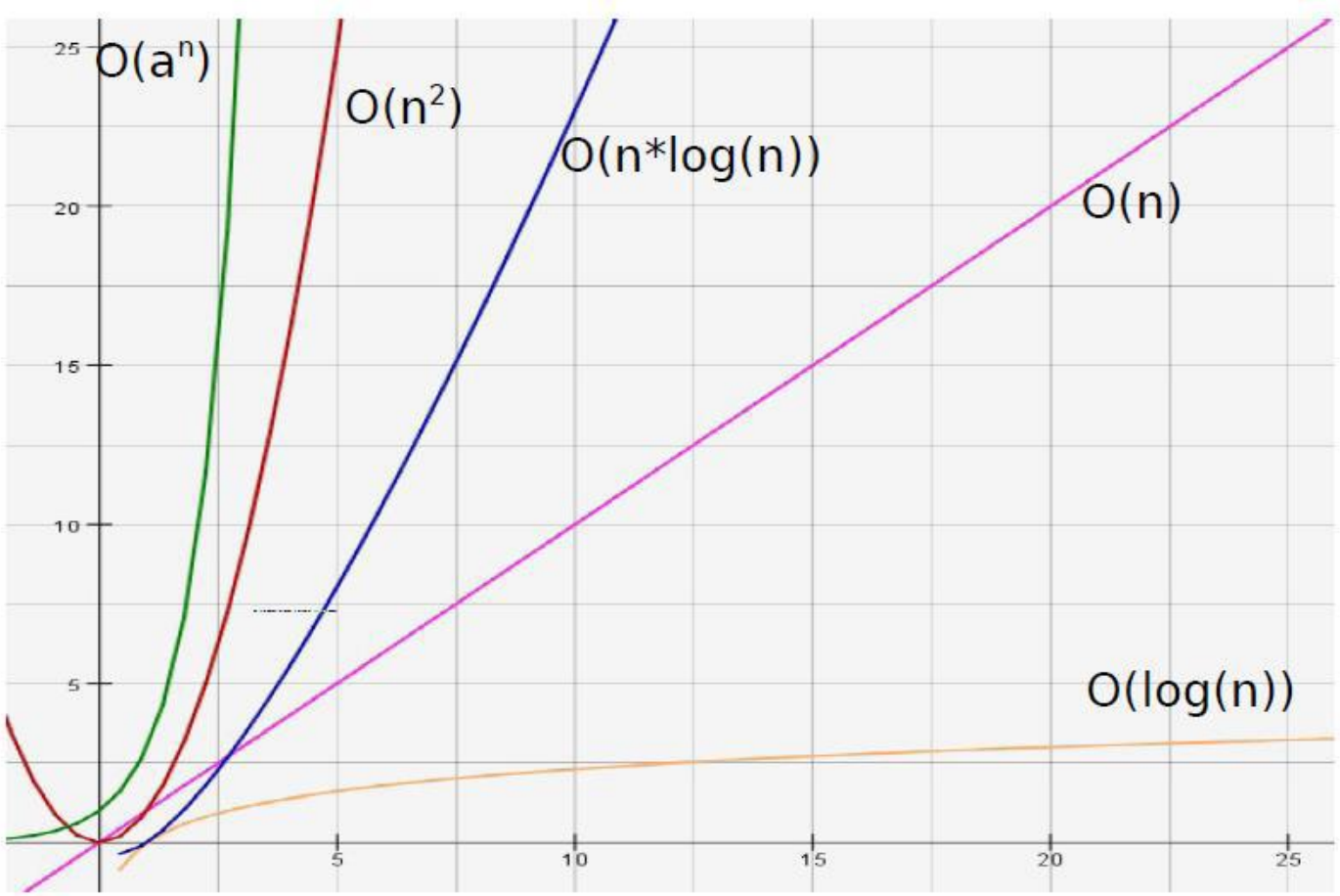
- Notation O :Propriétés
  - Si C(n) est un polynôme de degré d, alors C(n) est $O(n^d)$
    - On peut donc supprimer les termes de degré plus bas que d
    - On peut donc supprimer les facteurs constants
  - En pratique, on utilise la classe la plus basse possible
    - On dit que “2n est O(n)” si on a aussi “2n est $O(n^2)$
  - Et on utilise l’expression la plus simple
    - On dit que “3n+5 est O(n)” meme si on a aussi “3n+5 est O(3n)”

Classes de la complexité

❑**Classes de complexité**: On voit souvent apparaitre les complexités suivantes :

Complexité	Nom courant	Description
$O(1)$	constante	Le temps d'exécution ne dépend pas des données traitées, ce qui est assez rare !
$O(log(n))$	logarithmique	augmentation très faible du temps d'exécution quand le paramètre croit.
$O(n)$	linéaire	augmentation linéraire du temps d'exécution quand le paramètre croit.
$O(nlog(n))$	quasi-linéaire	augmentation un peu supérieure à O(n)
$O(n^2)$	quadratique	quand le paramètre double, le temps d'exécution est multiplié par 4. Exemple : algorithmes avec deux boucles imbriquées.
$O(n^k)$	polynômiale	ici, n^k est le terme de plus haut degré d'un polynôme en n ; il n'est pas rare de voir des complexités en $O(n^3)$ ou $O(n^4)$.
$O(k^n)$	exponentielle	quand le paramètre double, le temps d'exécution est élevé à la puissance k avec $k > 1$.
$O(n!)$	factorielle	asymptotiquement équivalente à n^n

Classes de la complexité



Autres notations

❑Grand Omega (borne asymptotique inférieure)

- C(n) est $\Omega\big(f(n)\big)$ s'il existe une constante $c > 0$ et un entier constant $n_0 \geq 1$ tels que pour tout $n \geq n_0$,

$$C(n) \geq c \times f(n) \; pour \; tout \; n \geq n_0$$

❑Grand-Theta(borne asymptotique approchée)

- C(n) est $\Theta\big(f(n)\big)$ s'il existe deux constantes c'>0 et c''>0 et un entier constant $n_0 \geq 1$ tels que

$$c' \times f(n) \leq C(n) \leq c'' \times f(n) \; pour \; tout \; n \geq n_0$$

- En notation grand-omega et grand-theta, on omet également les constantes et les termes de degré inférieur

Signification asymptotique des notations

- Grand O
  - $C(n)$ est $O\big(f(n)\big)$ si C$(\boldsymbol{n})$ est asymptotiquement plus petit ou égal à $f(n)$
- Grand-Omega
  - $C(n)$ est $\Omega\big(f(n)\big)$ si $C(n)$ est asymptotiquement plus grand ou égal à $f(n)$
- Grand-Theta
  - $C(n)$ est $\Theta\big(f(n)\big)$ si $C(n)$ est asymptotiquement égal à $f(n)$
  - Si $C(n)$ est $\boldsymbol{O}(f(n))$ et $\Omega\big(f(n)\big)$ alors C$(\boldsymbol{n})$ est $\Theta\big(f(n)\big)$

Signification asymptotique des notations

- Exemple de Grand-O:
  - $7n - 2$ est $O(n)$

Signification asymptotique des notations

❑Exemple de Grand-O:

- $7n - 2$ est $O(n)$
  - On veut $c > 0$ et $n_0 \geq 1$ tels que $7n - 2 \leq c \times n$ pour $n \geq n_0$
  - C'est vrai pour $c = 7$ et $n_0 = 1$

Signification asymptotique des notations

❑Exemple de Grand-O:

- $7n - 2$ est $O(n)$
  - On veut $c > 0$ et $n_0 \geq 1$ tels que $7n - 2 \leq c \times n$ pour $n \geq n_0$
  - C'est vrai pour $c = 7$ et $n_0 = 1$
- $3n^3 + 20n^2 + 5$ est $O(n^3)$

Signification asymptotique des notations

❑Exemple de Grand-O:

- $7n - 2$ est $O(n)$
  - On veut $c > 0$ et $n_0 \geq 1$ tels que $7n - 2 \leq c \times n$ pour $n \geq n_0$
  - C'est vrai pour $c = 7$ et $n_0 = 1$
- $3n^3 + 20n^2 + 5$ est $O(n^3)$
  - On veut $c > 0$ et $n_0 \geq 1$ tels que $3n^3 + 20n^2 + 5 \leq c \times n^3$ pour $n \geq n_0$
  - C'est vrai pour $c = 4$ et $n_0 = 21$

Signification asymptotique des notations

❑Exemple de Grand-O:

- $7n - 2$ est $O(n)$
  - On veut $c > 0$ et $n_0 \geq 1$ tels que $7n - 2 \leq c \times n$ pour $n \geq n_0$
  - C'est vrai pour $c = 7$ et $n_0 = 1$
- $3n^3 + 20n^2 + 5$ est $O(n^3)$
  - On veut $c > 0$ et $n_0 \geq 1$ tels que $3n^3 + 20n^2 + 5 \leq c \times n^3$ pour $n \geq n_0$
  - C'est vrai pour $c = 4$ et $n_0 = 21$
- $3 \log n + 5$ est $O(\log n)$

Signification asymptotique des notations

❑Exemple de Grand-O:

- $7n - 2$ est $O(n)$
  - On veut $c > 0$ et $n_0 \geq 1$ tels que $7n - 2 \leq c \times n$ pour $n \geq n_0$
  - C'est vrai pour $c = 7$ et $n_0 = 1$
- $3n^3 + 20n^2 + 5$ est $O(n^3)$
  - On veut $c > 0$ et $n_0 \geq 1$ tels que $3n^3 + 20n^2 + 5 \leq c \times n^3$ pour $n \geq n_0$
  - C'est vrai pour $c = 4$ et $n_0 = 21$
- $3 \log n + 5$ est $O(\log n)$
  - On veut $c > 0$ et $n_0 \geq 1$ tels que $3 \log n + 5 \leq c \times \log n$ pour $n \geq n_0$
  - C'est vrai pour $c = 8$ et $n_0 = 10$

Signification asymptotique des notations

❑Exemple de Ω et Θ :

- $5n^2$ est $\mathbf{\Omega}(n^2)$

Signification asymptotique des notations

❑Exemple de Ω et Θ :

- $5n^2$ est $\mathbf{\Omega}(n^2)$
  - $f(n)$ est $\mathbf{\Omega}(g(n))$ s'il existe une constante $c > 0$ et un entier constant $n_0 \geq 1$ tels que $f(n) \geq c \times g(n)$ pour tout $n \geq n_0$
  - Vrai pour $c = 5$ et $n_0 = 1$

Signification asymptotique des notations

❑Exemple de Ω et Θ :

- $5n^2$ est $\mathbf{\Omega}(n^2)$
  - $f(n)$ est $\mathbf{\Omega}(g(n))$ s'il existe une constante $c > 0$ et un entier constant $n_0 \geq 1$ tels que $f(n) \geq c \times g(n)$ pour tout $n \geq n_0$
  - Vrai pour $c = 5$ et $n_0 = 1$
- $5n^2$ est $\mathbf{\Omega}(n)$

Signification asymptotique des notations

❑Exemple de Ω et Θ :

- $5n^2$ est $\mathbf{\Omega}(n^2)$
  - $f(n)$ est $\mathbf{\Omega}(g(n))$ s'il existe une constante $c > 0$ et un entier constant $n_0 \geq 1$ tels que $f(n) \geq c \times g(n)$ pour tout $n \geq n_0$
  - Vrai pour $c = 5$ et $n_0 = 1$
- $5n^2$ est $\mathbf{\Omega}(n)$
  - $f(n)$ est $\mathbf{\Omega}(g(n))$ s'il existe une constante $c > 0$ et un entier constant $n_0 \geq 1$ tels que $f(n) \geq c \times g(n)$ pour tout $n \geq n_0$
  - Vrai pour $c = 1$ et $n_0 = 1$

Signification asymptotique des notations

❑Exemple de Ω et Θ :

- $5n^2$ est $\boldsymbol{\Omega}(n^2)$
  - $f(n)$ est $\boldsymbol{\Omega}(g(n))$ s'il existe une constante $c > 0$ et un entier constant $n_0 \geq 1$ tels que $f(n) \geq c \times g(n)$ pour tout $n \geq n_0$
  - Vrai pour $c = 5$ et $n_0 = 1$
- $5n^2$ est $\boldsymbol{\Omega}(n)$
  - $f(n)$ est $\boldsymbol{\Omega}(g(n))$ s'il existe une constante $c > 0$ et un entier constant $n_0 \geq 1$ tels que $f(n) \geq c \times g(n)$ pour tout $n \geq n_0$
  - Vrai pour $c = 1$ et $n_0 = 1$
- $5n^2$ est $\Theta(n^2)$

Signification asymptotique des notations

❑Exemple de Ω et Θ :

- $5n^2$ est $\boldsymbol{\Omega}(n^2)$
  - $f(n)$ est $\boldsymbol{\Omega}(g(n))$ s'il existe une constante $c > 0$ et un entier constant $n_0 \geq 1$ tels que $f(n) \geq c \times g(n)$ pour tout $n \geq n_0$
  - Vrai pour $c = 5$ et $n_0 = 1$
- $5n^2$ est $\boldsymbol{\Omega}(n)$
  - $f(n)$ est $\boldsymbol{\Omega}(g(n))$ s'il existe une constante $c > 0$ et un entier constant $n_0 \geq 1$ tels que $f(n) \geq c \times g(n)$ pour tout $n \geq n_0$
  - Vrai pour $c = 1$ et $n_0 = 1$
- $5n^2$ est $\Theta(n^2)$
  - $f(n)$ est $\Theta(g(n))$ s'il est $\boldsymbol{\Omega}(n^2)$ et $O(n^2)$. On a déjà vu le premier. Pour le second, $f(n)$ est $O(g(n))$ s'il existe une constante $c > 0$ et un entier constant $n_0 \geq 1$ tels que $f(n) \leq c \times g(n)$ pour tout $n \geq n_0$
  - Vrai pour $c = 5$ et $n_0 = 1$